1. List all the namespaces in the cluster

Solution:

```
kubectl get namespaces
kubectl get ns
```

2. List all the pods in all namespaces

Solution:

```
kubectl get po --all-namespaces
```

3. List all the pods in the particular namespace

Solution:

```
kubectl get po -n <namespace name>
```

4. List all the services in the particular namespace

Solution:

```
kubectl get svc -n <namespace name>
```

5. List all the pods showing name and namespace with a json path expression

Solution:

```
kubectl get pods -o=jsonpath="{.items[*]['metadata.name', 'metadata.namespace']}"
```

6. Create an nginx pod in a default namespace and verify the pod running

Solution:

```
// creating a pod
kubectl run nginx --image=nginx --restart=Never
// List the pod
kubectl get po
```

7. generate the yaml for pod called nginx2 & write to /opt/nginx203.yml. DONOT create the pod

Solution:

```
// get the yaml file with --dry-run flag
kubectl run nginx2 --image=nginx --dry-run -o yaml > /opt/nginx203.yml
```

8. Output the yaml file of the pod nginx created above & write the output to /opt/nginx.yml

Solution:

```
kubectl get po nginx -o yaml > /opt/nginx.yml
```

9. Output the yaml file of the pod you just created without the cluster-specific information

Solution:

```
kubectl get po nginx -o yaml --export
```

10. Get the complete details of the pod you just created

Solution:

```
kubectl describe pod nginx
```

11. Delete the pod you just created

Solution:

```
kubectl delete po nginx
kubectl delete -f nginx-pod.yaml
```

12. create a pod named alpine with image nginx & Delete the pod created without any delay (force delete)

Solution:

```
kubectl delete po alpine --grace-period=0 --force
```

13. Create the nginx pod with version 1.17.4 and expose it on port 80

Solution:

```
kubectl run nginx --image=nginx:1.17.4 --restart=Never --port=80
```

14. Change the Image version to 1.15-alpine for the pod you just created and verify the image version is updated

Solution:

```
kubectl set image pod/nginx nginx=nginx:1.15-alpine
kubectl describe po nginx
// another way it will open vi editor and change the version
kubeclt edit po nginx
kubectl describe po nginx
```

15. Change the Image version back to 1.17.1 for the pod you just updated and observe the changes

Solution:

```
kubectl set image pod/nginx nginx=nginx:1.17.1
kubectl describe po nginx
kubectl get po nginx -w # watch it
```

16. Check the Image version without the describe command

Solution:

```
kubectl get po nginx -o jsonpath='{.spec.containers[].image}{"\n"}'
```

17. Create the nginx pod and execute the simple shell on the pod

Solution:

```
// creating a pod
kubectl run nginx --image=nginx --restart=Never
```

// exec into the pod
kubectl exec -it nginx /bin/sh

18. Get the IP Address of the pod you just created

Solution:

kubectl get po nginx -o wide

19. Create a busybox pod and run command ls while creating it and check the logs

Solution:

kubectl run busybox --image=busybox --restart=Never -- ls
kubectl logs busybox

20. If pod crashed check the previous logs of the pod

Solution:

kubectl logs busybox -p

21. Create a busybox pod with command sleep 3600

Solution:

kubectl run busybox --image=busybox --restart=Never -- /bin/sh -c "sleep 3600"

22. Check the connection of the nginx pod from the busybox pod

Solution:

kubectl get po nginx -o wide
// check the connection
kubectl exec -it busybox -- wget -o- <IP Address>

23. Create a busybox pod and echo message 'How are you' and delete it manually

Solution:

kubectl run busybox --image=nginx --restart=Never -it -- echo "How are you"
kubectl delete po busybox

24. Create a busybox pod and echo message 'How are you' and have it deleted immediately

Solution:

// notice the --rm flag
kubectl run busybox --image=nginx --restart=Never -it --rm -- echo "How are you"

25. Create an nginx pod and list the pod with different levels of verbosity

Solution:

// create a pod
kubectl run nginx --image=nginx --restart=Never --port=80
// List the pod with different verbosity
kubectl get po nginx --v=7

kubectl get po nginx --v=8
kubectl get po nginx --v=9

26. List the nginx pod with custom columns POD_NAME and POD_STATUS

Solution:

kubectl get po -o=custom-columns="POD_NAME:.metadata.name, POD_STATUS:.status.containerStatuses[].state"

27. List all the pods sorted by name

Solution:

kubectl get pods --sort-by=.metadata.name

28. List all the pods sorted by created timestamp

Solution:

kubectl get pods--sort-by=.metadata.creationTimestamp

29. Create a Pod with three busy box containers with commands “ls; sleep 3600;”, “echo Hello World; sleep 3600;” and “echo this is the third container; sleep 3600” respectively and check the status

Solution:

// first create single container pod with dry run flag

kubectl run busybox --image=busybox --restart=Never --dry-run=client -o yaml -- bin/sh -c "sleep 3600; ls" > multi-container.yaml

// edit the pod to following yaml and create it

kubectl create -f multi-container.yaml

kubectl get po busybox

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
        run: busybox
  name: busybox
spec:
  containers:
  - args:
        - bin/sh
        - -c
        - ls; sleep 3600
        image: busybox
        name: busybox1
        resources: {}
```

```
    - args:
        - bin/sh
        - -c
        - echo Hello world; sleep 3600
        image: busybox
        name: busybox2
        resources: {}
    - args:
        - bin/sh
        - -c
        - echo this is third container; sleep 3600
        image: busybox
        name: busybox3
        resources: {}
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}
```

30. Check the logs of each container that you just created

    Solution:

    kubectl logs busybox -c busybox1

    kubectl logs busybox -c busybox2

    kubectl logs busybox -c busybox3

31. Check the previous logs of the second container busybox2 if any

    Solution:

    kubectl logs busybox -c busybox2 --previous

32. Run command ls in the third container busybox3 of the above pod

    Solution:

    kubectl exec busybox -c busybox3 -- ls

33. Show metrics of the above pod containers and puts them into the file.log and verify

    Solution:

    kubectl top pod busybox --containers

    // putting them into file

    kubectl top pod busybox --containers > file.log

    cat file.log

34. Create a Pod with main container busybox and which executes this “while true; do echo ‘Hi I am from Main container’ >> /var/log/index.html; sleep 5; done” and with sidecar container with nginx image which exposes on port 80. Use emptyDir Volume and mount this volume on path /var/log for busybox and on path /usr/share/nginx/html for nginx container. Verify both containers are running.

Solution:

// create an initial yaml file with this

kubectl run multi-cont-pod --image=busbox --restart=Never --dry-run -o yaml > multi-container.yaml

// edit the yml as below and create it

kubectl create -f multi-container.yaml

kubectl get po multi-cont-pod

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: multi-cont-pod
  name: multi-cont-pod
spec:
  volumes:
  - name: var-logs
    emptyDir: {}
  containers:
  - image: busybox
    command: ["/bin/sh"]
    args: ["-c", "while true; do echo 'Hi I am from Main container' >> /var/log/index.html; sleep 5;done"]
    name: main-container
    resources: {}
    volumeMounts:
    - name: var-logs
      mountPath: /var/log
  - image: nginx
    name: sidecar-container
    resources: {}
    ports:
      - containerPort: 80
    volumeMounts:
    - name: var-logs
      mountPath: /usr/share/nginx/html
```

```
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

35. Exec into both containers and verify that main.txt exist and query the main.txt from sidecar container with curl localhost

Solution:

```
// exec into main container
kubectl exec -it  multi-cont-pod -c main-container -- sh
cat /var/log/main.txt
// exec into sidecar container
kubectl exec -it  multi-cont-pod -c sidecar-container -- sh
cat /usr/share/nginx/html/index.html
// install curl and get default page
kubectl exec -it  multi-cont-pod -c sidecar-container -- sh
# apt-get update && apt-get install -y curl
# curl localhost
```

36. Get the pods with label information

Solution:

```
kubectl get pods --show-labels
```

37. Create 5 nginx pods in which two of them is labeled env=prod and three of them is labeled env=dev

Solution:

```
kubectl run nginx-dev1 --image=nginx --restart=Never --labels=env=dev
kubectl run nginx-dev2 --image=nginx --restart=Never --labels=env=dev
kubectl run nginx-dev3 --image=nginx --restart=Never --labels=env=dev
kubectl run nginx-prod1 --image=nginx --restart=Never --labels=env=prod
kubectl run nginx-prod2 --image=nginx --restart=Never --labels=env=prod
```

38. Verify all the pods are created with correct labels

Solution:

```
kubeclt get pods --show-labels
```

39. Get the pods with label env=dev

Solution:

```
kubectl get pods -l env=dev
```

40. Get the pods with label env=dev and also output the labels

Solution:

```
kubectl get pods -l env=dev --show-labels
```

41. Get the pods with label env=prod

Solution:

```
kubectl get pods -l env=prod
```

42. Get the pods with label env=prod and also output the labels

Solution:

```
kubectl get pods -l env=prod --show-labels
```

43. Get the pods with label env

Solution:

```
kubectl get pods -L env
```

44. Get the pods with labels env=dev and env=prod

Solution:

```
kubectl get pods -l 'env in (dev,prod)'
```

45. Get the pods with labels env=dev and env=prod and output the labels as well

Solution:

```
kubectl get pods -l 'env in (dev,prod)' --show-labels
```

46. Change the label for one of the pod to env=uat and list all the pods to verify

Solution:

```
kubectl label pod/nginx-dev3 env=uat --overwrite
kubectl get pods --show-labels
```

47. Remove the labels for the pods that we created now and verify all the labels are removed

Solution:

```
kubectl label pod nginx-dev{1..3} env-
kubectl label pod nginx-prod{1..2} env-
kubectl get po --show-labels
```

48. Let's add the label app=nginx for all the pods and verify

Solution:

```
kubectl label pod nginx-dev{1..3} app=nginx
kubectl label pod nginx-prod{1..2} app=nginx
kubectl get po --show-labels
```

49. Get all the nodes with labels (if using minikube you would get only master node)

Solution:

```
kubectl get nodes --show-labels
```

Learn & Lead IT services

50. Label the node node1 nodeName=nginxnode

Solution:

kubectl label node node1 nodeName=nginxnode

51. Create a Pod that will be deployed on this node with the label nodeName=nginxnode

Solution:

kubectl run nginx --image=nginx --restart=Never --dry-run=client -o yaml > pod.yaml

// add the nodeSelector like below and create the pod

kubectl create -f pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
        run: nginx
  name: nginx
spec:
  nodeSelector:
        nodeName: nginxnode
  containers:
  - image: nginx
        name: nginx
        resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

52. Verify the pod that it is scheduled with the node selector

Solution:

kubectl describe po nginx | grep Node-Selectors

53. Verify the pod nginx that we just created has this label

Solution:

kubectl describe po nginx | grep Labels

==========

54. Annotate the pods with name=webapp

Solution:

kubectl annotate pod nginx-dev{1..3} name=webapp

kubectl annotate pod nginx-prod{1..2} name=webapp

55. Verify the pods that have been annotated correctly

Solution:

```
kubectl describe po nginx-dev{1..3} | grep -i annotations
kubectl describe po nginx-prod{1..2} | grep -i annotations
```

56. Remove the annotations on the pods and verify

Solution:

```
kubectl annotate pod nginx-dev{1..3} name-
kubectl annotate pod nginx-prod{1..2} name-
kubectl describe po nginx-dev{1..3} | grep -i annotations
kubectl describe po nginx-prod{1..2} | grep -i annotations
```

==========

57. Remove all the pods that we created so far

Solution:

```
kubectl delete po --all
```

58. Create a deployment called webapp with image nginx with 5 replicas

Solution:

```
kubectl create deploy webapp --image=nginx --dry-run -o yaml > webapp.yaml
// change the replicas to 5 in the yaml and create it
kubectl create -f webapp.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: webapp
  name: webapp
spec:
  replicas: 5
  selector:
    matchLabels:
      app: webapp
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: webapp
    spec:
```

```
        containers:
        - image: nginx
            name: nginx
            resources: {}
    status: {}
```

59. Get the deployment you just created with labels

    Solution:

    kubectl get deploy webapp --show-labels

60. Output the yaml file of the deployment you just created

    Solution:

    kubectl get deploy webapp -o yaml

61. Get the pods of this deployment

    Solution:

    // get the label of the deployment
    kubectl get deploy --show-labels
    // get the pods with that label
    kubectl get pods -l app=webapp

62. Scale the deployment from 5 replicas to 8 replicas and verify

    Solution:

    kubectl scale deploy webapp --replicas=8
    kubectl get po -l app=webapp

63. Get the deployment rollout status

    Solution:

    kubectl rollout status deploy webapp

64. Get the replicaset that created with this deployment

    Solution:

    kubectl get rs -l app=webapp

65. Get the yaml of the replicaset and pods of this deployment

    Solution:

    kubectl get rs -l app=webapp -o yaml
    kubectl get po -l app=webapp -o yaml

66. Delete the deployment you just created and watch all the pods are also being deleted

    Solution:

    kubectl delete deploy webapp

Learn & Lead IT services

```
kubectl get po -l app=webapp -w
```

67. Create a deployment of webapp with image nginx:1.17.1 with container port 80 and verify the image version

Solution:

```
kubectl create deploy webapp --image=nginx:1.17.1 --dry-run -o yaml > webapp.yaml
// add the port section and create the deployment
kubectl create -f webapp.yaml
// verify
kubectl describe deploy webapp | grep Image
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: webapp
  name: webapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: webapp
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: webapp
    spec:
      containers:
      - image: nginx:1.17.1
        name: nginx
        ports:
        - containerPort: 80
        resources: {}
status: {}
```

68. Update the deployment with the image version 1.17.4 and verify

Solution:

```
kubectl set image deploy/webapp nginx=nginx:1.17.4
kubectl describe deploy webapp | grep Image
```

69. Check the rollout history and make sure everything is ok after the update

Solution:

kubectl rollout history deploy webapp

kubectl get deploy webapp --show-labels

kubectl get rs -l app=webapp

kubectl get po -l app=webapp

70. Undo the deployment to the previous version 1.17.1 and verify Image has the previous version

Solution:

kubectl rollout undo deploy webapp

kubectl describe deploy webapp | grep Image

71. Update the deployment with the image version 1.16.1 and verify the image and also check the rollout history

Solution:

kubectl set image deploy/webapp nginx=nginx:1.16.1

kubectl describe deploy webapp | grep Image

kubectl rollout history deploy webapp

72. Update the deployment to the Image 1.17.1 and verify everything is ok

Solution:

kubectl rollout undo deploy webapp --to-revision=3

kubectl describe deploy webapp | grep Image

kubectl rollout status deploy webapp

73. Update the deployment with the wrong image version 1.100 and verify something is wrong with the deployment

Solution:

kubectl set image deploy/webapp nginx=nginx:1.100

kubectl rollout status deploy webapp (still pending state)

kubectl get pods (ImagePullErr)

74. Undo the deployment with the previous version and verify everything is Ok

Solution:

kubectl rollout undo deploy webapp

kubectl rollout status deploy webapp

kubectl get pods

75. Check the history of the specific revision of that deployment

Solution:

kubectl rollout history deploy webapp --revision=7

76. Pause the rollout of the deployment

Solution:

kubectl rollout pause deploy webapp

77. Update the deployment with the image version latest and check the history and verify nothing is going on

Solution:

kubectl set image deploy/webapp nginx=nginx:latest

kubectl rollout history deploy webapp (No new revision)

78. Resume the rollout of the deployment

Solution:

kubectl rollout resume deploy webapp

79. Check the rollout history and verify it has the new version

Solution:

kubectl rollout history deploy webapp

kubectl rollout history deploy webapp --revision=9

80. Apply the autoscaling to this deployment with minimum 10 and maximum 20 replicas and target CPU of 85% and verify hpa is created and replicas are increased to 10 from 1

Solution:

kubectl autoscale deploy webapp --min=10 --max=20 --cpu-percent=85

kubectl get hpa

kubectl get pod -l app=webapp

81. Clean the cluster by deleting deployment and hpa you just created

Solution:

kubectl delete deploy webapp

kubectl delete hpa webapp

82. Create a Job with an image node which prints node version and also verifies there is a pod created for this job

Solution:

kubectl create job nodeversion --image=node -- node -v

kubectl get job -w

kubectl get pod

83. Get the logs of the job just created

Solution:

kubectl logs <pod name> // created from the job

84.Output the yaml file for the Job with the image busybox which echos “Hello I am from job”

Solution:

kubectl create job hello-job --image=busybox --dry-run=client -o yaml -- echo "Hello I am from job"

85. Copy the above YAML file to hello-job.yaml file and create the job

Solution:

kubectl create job hello-job --image=busybox --dry-run=client -o yaml -- echo "Hello I am from job" > hello-job.yaml

kubectl create -f hello-job.yaml

86. Verify the job and the associated pod is created and check the logs as well

Solution:

kubectl get job

kubectl get po

kubectl logs hello-job-*

87. Delete the job we just created

Solution:

kubectl delete job hello-job

88. Create the same job and make it run 10 times one after one

Solution:

kubectl create job hello-job --image=busybox --dry-run -o yaml -- echo "Hello I am from job" > hello-job.yaml

// edit the yaml file to add completions: 10

kubectl create -f hello-job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  creationTimestamp: null
  name: hello-job
spec:
  completions: 10
  template:
        metadata:
          creationTimestamp: null
        spec:
          containers:
          - command:
```

```
                - echo
                - Hello I am from job
                image: busybox
                name: hello-job
                resources: {}
            restartPolicy: Never
        status: {}
```

89. Watch the job that runs 10 times one by one and verify 10 pods are created and delete those after it's completed

Solution:

```
kubectl get job -w
kubectl get po
kubectl delete job hello-job
```

90. Create the same job and make it run 10 times parallel

Solution:

```
kubectl create job hello-job --image=busybox --dry-run=client -o yaml -- echo "Hello I am from job" > hello-job.yaml
// edit the yaml file to add parallelism: 10
kubectl create -f hello-job.yaml
```

```
apiVersion: batch/v1
kind: Job
metadata:
  creationTimestamp: null
  name: hello-job
spec:
  parallelism: 10
  template:
      metadata:
        creationTimestamp: null
      spec:
        containers:
        - command:
            - echo
            - Hello I am from job
            image: busybox
            name: hello-job
            resources: {}
        restartPolicy: Never
status: {}
```

91. Watch the job that runs 10 times parallelly and verify 10 pods are created and delete those after it's completed

Solution:

kubectl get job -w

kubectl get po

kubectl delete job hello-job

92. Create a Cronjob with busybox image that prints date and hello from kubernetes cluster message for every minute

Solution:

kubectl create cronjob date-job --image=busybox --schedule="*/1 * * * *" -- bin/sh -c "date; echo Hello from kubernetes cluster"

93. Output the YAML file of the above cronjob

Solution:

kubectl get cj date-job -o yaml

94. Verify that CronJob creating a separate job and pods for every minute to run and verify the logs of the pod

Solution:

kubectl get job

kubectl get po

kubectl logs date-job-<jobid>-<pod>

95. Delete the CronJob and verify all the associated jobs and pods are also deleted.

Solution:

kubectl delete cj date-job

// verify pods and jobs

kubectl get po

kubectl get job

96. List Persistent Volumes in the cluster

Solution:

kubectl get pv

97. Create a hostPath PersistentVolume named task-pv-volume with storage 10Gi, access modes ReadWriteOnce, storageClassName manual, and volume at /mnt/data and verify

Solution:

kubectl create -f task-pv-volume.yaml

kubectl get pv

task-pv-volume.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
  labels:
        type: local
spec:
  storageClassName: manual
  capacity:
        storage: 10Gi
  accessModes:
        - ReadWriteOnce
  hostPath:
        path: "/mnt/data"
```

98. Create a PersistentVolumeClaim of at least 3Gi storage and access mode ReadWriteOnce and verify status is Bound

Solution:

kubectl create -f task-pv-claim.yaml

kubectl get pvc

task-pv-claim.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pv-claim
spec:
  storageClassName: manual
  accessModes:
        - ReadWriteOnce
  resources:
        requests:
          storage: 3Gi
```

99. Delete persistent volume and PersistentVolumeClaim we just created

Solution:

kubectl delete pvc task-pv-claim

kubectl delete pv task-pv-volume

100. Create a Pod with an image Redis and configure a volume that lasts for the lifetime of the Pod

Solution:

// emptyDir is the volume that lasts for the life of the pod

kubectl create -f redis-storage.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: redis
spec:
  containers:
  - name: redis
        image: redis
        volumeMounts:
        - name: redis-storage
          mountPath: /data/redis
  volumes:
  - name: redis-storage
        emptyDir: {}
```

101. Exec into the above pod and create a file named file.txt with the text 'This is called the file' in the path /data/redis and open another tab and exec again with the same pod and verifies file exist in the same path.

Solution:

```
// first terminal
kubectl exec -it redis-storage /bin/sh
cd /data/redis
echo 'This is called the file' > file.txt
//open another tab
kubectl exec -it redis-storage /bin/sh
cat /data/redis/file.txt
```

102. Delete the above pod and create again from the same yaml file and verifies there is no file.txt in the path /data/redis

Solution:

```
kubectl delete pod redis
kubectl create -f redis-storage.yaml
kubectl exec -it redis-storage /bin/sh
cat /data/redis/file.txt // file doesn't exist
```

103. Create PersistentVolume named task-pv-volume with storage 10Gi, access modes ReadWriteOnce, storageClassName manual, and volume at /mnt/data and Create a PersistentVolumeClaim of at least 3Gi storage and access mode ReadWriteOnce and verify status is Bound

Solution:

kubectl create -f task-pv-volume.yaml

```
kubectl create -f task-pv-claim.yaml
kubectl get pv
kubectl get pvc
```

104. Create an nginx pod with containerPort 80 and with a PersistentVolumeClaim task-pv-claim and has a mouth path "/usr/share/nginx/html"

Solution:

```
kubectl create -f task-pv-pod.yaml
task-pv-pod.yaml

apiVersion: v1
kind: Pod
metadata:
  name: task-pv-pod
spec:
  volumes:
        - name: task-pv-storage
          persistentVolumeClaim:
                claimName: task-pv-claim
  containers:
        - name: task-pv-container
          image: nginx
          ports:
                - containerPort: 80
                  name: "http-server"
          volumeMounts:
                - mountPath: "/usr/share/nginx/html"
                  name: task-pv-storage
```

105. List all the configmaps in the cluster

Solution:

```
kubectl get cm
        or
kubectl get configmap
```

106. Create a configmap called myconfigmap with literal value appname=myapp

Solution:

```
kubectl create cm myconfigmap --from-literal=appname=myapp
```

107. Verify the configmap we just created has this data

Solution:

```
// you will see under data
```

```
kubectl get cm -o yaml
                    or
kubectl describe cm
```

108. delete the configmap myconfigmap we just created

Solution:

```
kubectl delete cm myconfigmap
```

109. Create a file called config.txt with two values key1=value1 and key2=value2 and verify the file

Solution:

```
cat >> config.txt << EOF
key1=value1
key2=value2
EOF
cat config.txt
```

110. Create a configmap named keyvalcfgmap and read data from the file config.txt and verify that configmap is created correctly

Solution:

```
kubectl create cm keyvalcfgmap --from-file=config.txt
kubectl get cm keyvalcfgmap -o yaml
```

111. Create an nginx pod and load environment values from the above configmap keyvalcfgmap and exec into the pod and verify the environment variables store it on /opt/keyvalcfgmap-vars.txt. and delete the pod

Solution:

```
// first run this command to save the pod yml
kubectl run nginx --image=nginx --restart=Never --dry-run=client -o yaml > nginx-pod.yml
// edit the yml to below file and create
kubectl create -f nginx-pod.yml
// verify
kubectl exec nginx -- env > /opt/keyvalcfgmap-vars.txt
kubectl delete po nginx
```

nginx-pod.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
```

```
  labels:
        run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
        name: nginx
        resources: {}
        envFrom:
        - configMapRef:
              name: keyvalcfgmap
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

112. Create an env file file.env with var1=val1 and create a configmap envcfgmap from this env file and verify the configmap

Solution:

```
echo var1=val1 > file.env
cat file.env
kubectl create cm envcfgmap --from-env-file=file.env
kubectl get cm envcfgmap -o yaml --export
```

113. Create an nginx pod and load environment values from the above configmap envcfgmap and exec into the pod and verify the environment variables and delete the pod

Solution:

```
// first run this command to save the pod yml
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx-pod.yml
// edit the yml to below file and create
kubectl create -f nginx-pod.yml
// verify
kubectl exec -it nginx -- env
kubectl delete po nginx
```

nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
        run: nginx
```

```
  name: nginx
spec:
  containers:
  - image: nginx
      name: nginx
      resources: {}
      env:
      - name: ENVIRONMENT
        valueFrom:
          configMapKeyRef:
            name: envcfgmap
            key: var1
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

114. Create a configmap called cfgvolume with values var1=val1, var2=val2 and create an nginx pod with volume nginx-volume which reads data from this configmap cfgvolume and put it on the path /etc/cfg

Solution:

```
// first create a configmap cfgvolume
kubectl create cm cfgvolume --from-literal=var1=val1 --from-literal=var2=val2
// verify the configmap
kubectl describe cm cfgvolume
// create the config map
kubectl create -f nginx-volume.yml
// exec into the pod
kubectl exec -it nginx -- /bin/sh
// check the path
cd /etc/cfg
ls
```

nginx-volume.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  volumes:
  - name: nginx-volume
```

```
        configMap:
          name: cfgvolume
    containers:
    - image: nginx
        name: nginx
        resources: {}
        volumeMounts:
        - name: nginx-volume
          mountPath: /etc/cfg
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}
```

115. Create a pod called secbusybox with the image busybox which executes command sleep 3600 and makes sure any Containers in the Pod, all processes run with user ID 1000 and with group id 2000 and verify.

Solution:

```
// create yml file with dry-run
kubectl run secbusybox --image=busybox --restart=Never --dry-run -o yaml -- /bin/sh -c "sleep 3600;" > busybox.yml
// edit the pod like below and create
kubectl create -f busybox.yml
// verify
kubectl exec -it secbusybox -- sh
id // it will show the id and group
```

busybox.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: secbusybox
  name: secbusybox
spec:
  securityContext: # add security context
    runAsUser: 1000
    runAsGroup: 2000
  containers:
  - args:
    - /bin/sh
    - -c
```

```
        - sleep 3600;
        image: busybox
        name: secbusybox
        resources: {}
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}
```

116. Create the same pod as above this time set the securityContext for the container as well and verify that the securityContext of container overrides the Pod level securityContext.

Solution:

```
// create yml file with dry-run
kubectl run secbusybox --image=busybox --restart=Never --dry-run -o yaml -- /bin/sh -c "sleep 3600;" > busybox.yml
// edit the pod like below and create
kubectl create -f busybox.yml
// verify
kubectl exec -it secbusybox -- sh
id // you can see container securityContext overides the Pod level
```

busybox.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: secbusybox
  name: secbusybox
spec:
  securityContext:
    runAsUser: 1000
  containers:
  - args:
    - /bin/sh
    - -c
    - sleep 3600;
    image: busybox
    securityContext:
      runAsUser: 2000
    name: secbusybox
    resources: {}
```

```
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

117. Create pod with an nginx image and configure the pod with capabilities NET_ADMIN and SYS_TIME verify the capabilities

Solution:

```
// create the yaml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// edit as below and create pod
kubectl create -f nginx.yml
// exec and verify
kubectl exec -it nginx -- sh
cd /proc/1
cat status
// you should see these values
CapPrm: 00000000aa0435fb
CapEff: 00000000aa0435fb
```

nginx.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    securityContext:
      capabilities:
        add: ["SYS_TIME", "NET_ADMIN"]
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

118. Create a Pod nginx and specify a memory request and a memory limit of 100Mi and 200Mi respectively.

Solution:

```
// create a yml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// add the resources section and create
kubectl create -f nginx.yml
// verify
kubectl top pod
```

nginx.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources:
      requests:
        memory: "100Mi"
      limits:
        memory: "200Mi"
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

119. Create a Pod nginx and specify a CPU request and a CPU limit of 0.5 and 1 respectively.

Solution:

```
// create a yml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// add the resources section and create
kubectl create -f nginx.yml
// verify
kubectl top pod
```

nginx.yml

```
apiVersion: v1
kind: Pod
```

```
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources:
      requests:
        cpu: "0.5"
      limits:
        cpu: "1"
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

120. Create a Pod nginx and specify both CPU, memory requests and limits together and verify.

Solution:

```
// create a yml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// add the resources section and create
kubectl create -f nginx.yml
// verify
kubectl top pod
```

nginx.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources:
      requests:
        memory: "100Mi"
```

```
                    cpu: "0.5"
            limits:
                    memory: "200Mi"
                    cpu: "1"
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}
```

121. Create a Pod nginx and specify a memory request and a memory limit of 100Gi and 200Gi respectively which is too big for the nodes and verify pod fails to start because of insufficient memory

Solution:

```
// create a yml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// add the resources section and create
kubectl create -f nginx.yml
// verify
kubectl describe po nginx // you can see pending state
```

nginx.yml

```
  apiVersion: v1
  kind: Pod
  metadata:
    creationTimestamp: null
    labels:
        run: nginx
    name: nginx
  spec:
    containers:
    - image: nginx
        name: nginx
        resources:
          requests:
                memory: "100Gi"
                cpu: "0.5"
          limits:
                memory: "200Gi"
                cpu: "1"
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}
```

Learn & Lead IT services

122. Create a secret mysecret with values user=myuser and password=mypassword

Solution:

kubectl create secret generic my-secret --from-literal=username=user --from-literal=password=mypassword

123. List the secrets in all namespaces

Solution:

kubectl get secret --all-namespaces

124. Output the yaml of the secret created above

Solution:

kubectl get secret my-secret -o yaml

125. Create an nginx pod which reads username as the environment variable

Solution:

```
// create a yml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// add env section below and create
kubectl create -f nginx.yml
//verify
kubectl exec -it nginx -- env
```

nginx.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    env:
    - name: USER_NAME
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: username
    resources: {}
```

```
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

126. Create an nginx pod which loads the secret as environment variables

Solution:

```
// create a yml file
kubectl run nginx --image=nginx --restart=Never --dry-run -o yaml > nginx.yml
// add env section below and create
kubectl create -f nginx.yml
//verify
kubectl exec -it nginx -- env
```

nginx.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    envFrom:
    - secretRef:
        name: my-secret
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

127. List all the service accounts in the default namespace

Solution:

```
kubectl get sa
```

128. List all the service accounts in all namespaces

Solution:

```
kubectl get sa --all-namespaces
```

129. Create a service account called admin

Solution:

kubectl create sa admin

130. Output the YAML file for the service account we just created

Solution:

kubectl get sa admin -o yaml

131. Create a busybox pod which executes this command sleep 3600 with the service account admin and verify

Solution:

kubectl run busybox --image=busybox --restart=Never --dry-run -o yaml -- /bin/sh -c "sleep 3600" > busybox.yml

kubectl create -f busybox.yml

// verify

kubectl describe po busybox

busybox.yml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
        run: busybox
  name: busybox
spec:
  serviceAccountName: admin
  containers:
  - args:
        - /bin/sh
        - -c
        - sleep 3600
        image: busybox
        name: busybox
        resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

132. Create an nginx pod with containerPort 80 and it should only receive traffic only it checks the endpoint / on port 80 and verify and delete the pod.

Solution:

```
kubectl run nginx --image=nginx --restart=Never --port=80 --dry-run -o yaml > nginx-pod.yaml
// add the readinessProbe section and create
kubectl create -f nginx-pod.yaml
// verify
kubectl describe pod nginx | grep -i readiness
kubectl delete po nginx
```

nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    ports:
    - containerPort: 80
    readinessProbe:
      httpGet:
        path: /
        port: 80
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

133. Create an nginx pod with containerPort 80 and it should check the pod running at endpoint / healthz on port 80 and verify and delete the pod.

Solution:

```
kubectl run nginx --image=nginx --restart=Never --port=80 --dry-run -o yaml > nginx-pod.yaml
// add the livenessProbe section and create
kubectl create -f nginx-pod.yaml
// verify
kubectl describe pod nginx | grep -i readiness
kubectl delete po nginx
```

nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    ports:
    - containerPort: 80
    livenessProbe:
      httpGet:
        path: /healthz
        port: 80
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

134. Create an nginx pod with containerPort 80 and it should check the pod running at endpoint /healthz on port 80 and it should only receive traffic only it checks the endpoint / on port 80. verify the pod.

Solution:

```
kubectl run nginx --image=nginx --restart=Never --port=80 --dry-run -o yaml > nginx-pod.yaml
// add the livenessProbe and readiness section and create
kubectl create -f nginx-pod.yaml
// verify
kubectl describe pod nginx | grep -i readiness
kubectl describe pod nginx | grep -i liveness
```

nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: nginx
  name: nginx
```

```
spec:
  containers:
  - image: nginx
        name: nginx
        ports:
        - containerPort: 80
        livenessProbe:
          httpGet:
              path: /healthz
              port: 80
        readinessProbe:
          httpGet:
              path: /
              port: 80
        resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

135. Check what all are the options that we can configure with readiness and liveness probes

Solution:

kubectl explain Pod.spec.containers.livenessProbe

kubectl explain Pod.spec.containers.readinessProbe

136. Create the pod nginx with the above liveness and readiness probes so that it should wait for 20 seconds before it checks liveness and readiness probes and it should check every 25 seconds.

Solution:

kubectl create -f nginx-pod.yaml

nginx-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
        run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
        name: nginx
        ports:
```

```
        - containerPort: 80
        livenessProbe:
          initialDelaySeconds: 20
          periodSeconds: 25
          httpGet:
            path: /healthz
            port: 80
        readinessProbe:
          initialDelaySeconds: 20
          periodSeconds: 25
          httpGet:
            path: /
            port: 80
        resources: {}
    dnsPolicy: ClusterFirst
    restartPolicy: Never
  status: {}
```

137. Create a busybox pod with this command “echo I am from busybox pod; sleep 3600;” and verify the logs.

Solution:

kubectl run busybox --image=busybox --restart=Never -- /bin/sh -c "echo I am from busybox pod; sleep 3600;"

kubectl logs busybox

138. copy the logs of the above pod to the busybox-logs.txt and verify

Solution:

kubectl logs busybox > busybox-logs.txt

cat busybox-logs.txt

139. List all the events sorted by timestamp and put them into file.log and verify

Solution:

kubectl get events --sort-by=.metadata.creationTimestamp

// putting them into file.log

kubectl get events --sort-by=.metadata.creationTimestamp > file.log

cat file.log

140. Create a pod with an image alpine which executes this command ”while true; do echo ‘Hi I am from alpine’; sleep 5; done” and verify and follow the logs of the pod.

Solution:

// create the pod

kubectl run hello --image=alpine --restart=Never -- /bin/sh -c "while true; do echo 'Hi I am from Alpine'; sleep 5;done"

// verify and follow the logs

kubectl logs --follow hello

141. Create the pod with this kubectl create -f https://raw.githubusercontent.com/lerndevops/educka/master/exam-prep/not-running.yml The pod is not in the running state. Debug it.

Solution:

// create the pod

kubectl create -f https://raw.githubusercontent.com/lerndevops/educka/master/exam-prep/not-running.yml

// get the pod

kubectl get pod not-running

kubectl describe po not-running

// it clearly says ImagePullBackOff something wrong with image

kubectl edit pod not-running // it will open vim editor

or

kubectl set image pod/not-running not-running=redis

142. This following yaml creates 4 namespaces and 4 pods. One of the pod in one of the namespaces are not in the running state. Debug and fix it. https://raw.githubusercontent.com/lerndevops/educka/master/exam-prep/problem-pod.yml

Solution:

kubectl create -f https://raw.githubusercontent.com/lerndevops/educka/master/exam-prep/problem-pod.yml

// get all the pods in all namespaces

kubectl get po --all-namespaces

// find out which pod is not running

kubectl get po -n namespace2

// update the image

kubectl set image pod/pod2 pod2=nginx -n namespace2

// verify again

kubectl get po -n namespace2

143. Get the memory and CPU usage of all the pods and find out top 3 pods which have the highest usage and put them into the cpu-usage.txt file

Solution:

// get the top 3 hungry pods

kubectl top pod --all-namespaces | sort --reverse --key 3 --numeric | head -3

```
// putting into file
kubectl top pod --all-namespaces | sort --reverse --key 3 --numeric | head -3 > cpu-usage.txt
// verify
cat cpu-usage.txt
```

144. Create an nginx pod with a yaml file with label my-nginx and expose the port 80

Solution:

```
kubectl run nginx --image=nginx --restart=Never --port=80 --dry-run -o yaml > nginx.yaml
// edit the label app: my-nginx and create the pod
kubectl create -f nginx.yaml
```

nginx.yaml

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    app: my-nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    ports:
    - containerPort: 80
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

145. Create the service for this nginx pod with the pod selector app: my-nginx

Solution:

```
// create the below service
kubectl create -f nginx-svc.yaml
```

nginx-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
```

```
spec:
  selector:
        app: my-nginx
  ports:
        - protocol: TCP
          port: 80
          targetPort: 9376
```

146. Find out the label of the pod and verify the service has the same label

Solution:

```
// get the pod with labels
kubectl get po nginx --show-labels
// get the service and chekc the selector column
kubectl get svc my-service -o wide
```

147. Delete the service and create the service with kubectl expose command and verify the label

Solution:

```
// delete the service
kubectl delete svc my-service
// create the service again
kubectl expose po nginx --port=80 --target-port=9376
// verify the label
kubectl get svc -l app=my-nginx
```

148. Delete the service and create the service again with type NodePort

Solution:

```
// delete the service
kubectl delete svc nginx
// create service with expose command
kubectl expose po nginx --port=80 --type=NodePort
```

149. Create the temporary busybox pod and hit the service. Verify the service that it should return the nginx page index.html.

Solution:

```
// get the clusterIP from this command
kubectl get svc nginx -o wide
// create temporary busybox to check the nodeport
kubectl run busybox --image=busybox --restart=Never -it --rm -- wget -o- <Cluster IP>:80
```

150. Create a NetworkPolicy which denies all ingress traffic

Solution:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

151. Create a yaml file called db-secret.yaml for a secret called db-user-pass. The secret should have two fields: a username and password. The username should be "superadmin" and the password should be "imamazing".

Solution:

```
echo -n 'superadmin' > ./username.txt
echo -n 'imamazing' > ./password.txt

kc secret generic db-user-pass --from-file=./username.txt --from-file=./password.txt -o yaml > db-secret.yaml
```

Learn & Lead IT services

```
apiVersion: v1
data:
  password.txt: aWhlYXJ0a2l0dGVucw==
  username.txt: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: 2019-03-03T00:21:16Z
  name: db-user-pass
  namespace: default
  resourceVersion: "30182"
  selfLink: /api/v1/namespaces/default/secrets/db-user-pass
  uid: 42b979da-3d4a-11e9-8f41-06f514f6b3f0
type: Opaque
```

152. Create a ConfigMap called web-config that contains the following two entries: 'web_port' set to 'localhost:8080' 'external_url' set to 'reddit.com' Run a pod called web-config-pod running nginx, expose the configmap settings as environment variables inside the nginx container.

Solution:

this has to be done in several steps, first create configmap from literals on command line

sudo kubectl create configmap test-cm --from-literal=web_port='localhost:8080' --from-literal=external_url='reddit.com'

double check the configmap

sudo kubectl describe cm test-cm

create the pod deployment yaml and then edit the file

sudo kubectl run web-config-pod --image=nginx --dry-run=client -o yaml > web-config-pod.yaml

edit the file: vi web-config-pod.yaml

```
  spec:
        containers:
        - image: nginx
              env:
               - name: WEB_PORT
                 valueFrom:
                        configMapKeyRef:
                          name: test-cm
                          key: web_port
               - name: EXTERNAL_URL
                 valueFrom:
                        configMapKeyRef:
                          name: test-cm
                          key: external_url
```

sudo kubectl create -f web-config-pod.yaml

get env vars from nginx pod:

kubectl exec nginx-deploy-868c8d4b79-czc2j env

153. list all the pods with Running status in the default namespace & write the output to /opt/default-pod-status.txt

Solution:

kubectl get pods --sort-by=.status.phase -n default

154. list all the persistant volumes, sort by storage reqeust size & write the output to /opt/pv-size.txt

Solution:

kubectl get pvc --sort-by=.spec.resources.requests.storage

155. list all namespaces & sort by name

Solution:

kubectl get namespaces --sort-by=.metadata.name

156. Create a job that calculates pi to 2000 decimal points using the container with the image named perl and the following commands issued to the container: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]Once the job has completed, check the logs to and export the result to pi-result.txt.

Solution:

```
kubectl create job pi2000 --image=perl -o yaml --dry-run=client > pi2000.yaml
```

edit the file, edit the name, remove any ID references and include the command argument under container spec.

```
                    command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]

aapiVersion: batch/v1
kind: Job
metadata:
  creationTimestamp: null
  name: pi2000
spec:
  template:
        metadata:
          creationTimestamp: null
        spec:
          containers:
          - image: perl
                name: pi2000
                command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]
                resources: {}
          restartPolicy: Never
```

status: {}

kubectl create -f pi2000.yaml

get the output from the logs and export them to text file

kubectl logs pi2000-x247y > pi-result.txt

157. Create a yaml file called nginx-deploy.yaml for a deployment of three replicas of nginx, listening on the container's port 80. They should have the labels role=webserver and app=nginx. The deployment should be named nginx-deploy. Expose the deployment with a NodePort and use a curl statement on the IP address of the NodeIP
to export the output to a file titled output.txt.

Solution:

sudo kubectl create deployment nginx-deploy --image=nginx --dry-run=client -o yaml > nginx-deployment.yaml

edit the file nginx-deployment add the labels as below and modify the replicas to 3

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
        role: webserver    ## add
        app: nginx        ## add
  name: nginx-deploy
spec:
  replicas: 3    ## update to 3
  selector:
        matchLabels:
          role: webserver  ## add
          app: nginx       ## add
  template:
        metadata:
          labels:
                role: webserver  ## add
                app: nginx       ## add
        spec:
          containers:
          - image: nginx
```

```
name: nginx
ports:        ## add
  - containerPort: 80  ## add
```

expose the deployment with a NodePort type, call it nginx-service

kubectl expose deployment nginx-deploy --port 80 --type=NodePort --name=nginx-service

use a curl statement that connects to the IP endpoint of the nginx-service and save the output to a file called output.txt

curl NodeIP:NodePort > output.txt

158. Scale the deployment you just made down to 2 replicas

Solution:
sudo kubectl scale deployment nginx-deploy --replicas=2

159. Create a Deployment called "haz-docs" with an nginx image listening on port 80. Attach the pod to emptyDir storage, mounted to /tmp in the container. Connect to the pod and create a file with zero bytes in the /tmp directory called my-doc.txt.

Solution:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
        run: haz-docs
  name: haz-docs
spec:
  replicas: 1
  selector:
        matchLabels:
          run: haz-docs
  strategy: {}
  template:
        metadata:
          creationTimestamp: null
          labels:
```

```
                    run: haz-docs
            spec:
              containers:
              - image: nginx
                    name: haz-docs
                    volumeMounts:
                    - mountPath: /tmp
                      name: tmpvolume
                    ports:
                    - containerPort: 80
                    resources: {}
              volumes:
              - name: tmpvolume
                    emptyDir: {}
        status: {}
```

```
sudo kubectl exec -it haz-docs-5b49cb4d87-2lm5g /bin/bash

root@haz-docs-5b49cb4d87-2lm5g:/# cd /tmp/
root@haz-docs-5b49cb4d87-2lm5g:/tmp# touch my-doc.txt
root@haz-docs-5b49cb4d87-2lm5g:/tmp# ls
my-doc.txt
```

160. Label the worker node of your cluster with rack=qa.

Solution:

```
sudo kubectl label node texasdave2c.mylabserver.com rack=qa
```

161. Create a file called counter.yaml in your home directory and paste the following yaml into it:

```
apiVersion: v1
kind: Pod
metadata:
  name: counter
spec:
  containers:
  - name: count
    image: busybox
    args: [/bin/sh, -c, 'i=0; while true; do echo "$i: $(date)"; i=$((i+1)); sleep 1; done']
```

Start this pod.

Once its logs exceed a count of 20 (no need to be precise — any time after it has reached 20 is fine), save the logs into a file in your home directory called count.result.txt.

Solution:

```
vi counter.yaml

apiVersion: v1
kind: Pod
metadata:
  name: counter
spec:
  containers:
  - name: count
      image: busybox
      args: [/bin/sh, -c, 'i=0; while true; do echo "$i: $(date)"; i=$((i+1)); sleep 1; done']

kubectl create -f counter.yaml
kubectl logs counter > count.result.txt
```

162. Create a new namespace called "cloud9". Create a pod running nginx with a liveliness probe that uses httpGet to probe an endpoint path located at / on port 80. The initial delay is 3 seconds and the period is 3.

Solution:

```
kubectl create namespace cloud9

kubectl run nginx --image=nginx --port=80 --dry-run=client -o yaml > nginx-liveness.yml
vi nginx-liveness.yml

apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
      run: nginx
spec:
  containers:
  - name: nginx
      image: nginx
      ports:
```

```
        - containerPort: 80
        livenessProbe:     # add
    httpGet:        # add
      path: /       # add
      port: 80      # add
          initialDelaySeconds: 3    # add
          periodSeconds: 3         # add
```

kubectl create -f nginx-liveness.yml

163. Configure a DaemonSet to run the image k8s.gcr.io/pause:2.0 in the cluster.

Solution:

kubectl create deployment testds --image=k8s.gcr.io/pause:2.0 --dry-run=client -o yaml > testds.yaml

then edited it as Daemonset to get it running, you don't do replicas in a daemonset, it runs on all nodes

remove the replicas field from Yaml, ensure the Yaml looks like below.

```
apiVersion: apps/v1
kind: DaemonSet   ## update Deployment to DaemonSet
metadata:
  labels:
        app: testds
  name: testds
spec:
  selector:
        matchLabels:
          app: testds
  template:
        metadata:
          creationTimestamp: null
          labels:
                app: testds
        spec:
          containers:
          - image: k8s.gcr.io/pause:2.0
                name: pause
```

```
kubectl create -f testds.yaml
kubectl get ds  ## ensure the pods are running
```

164. An app inside a container needs the IP address of the web-dep endpoint to be passed to it as an environment variable called "ULTIMA". create a deployment with 1 replica using image nginx & assing the environment variable with values. Save the yaml as env-ultima.yaml

Solution:

```
### get the IP address of the web-dep service
kubectl create service clusterip web-dep --tcp=80:80
kubectl get svc web-dep   ## note the clusterIP
```

```
kubectl create deployment ultima-dep --dry-run=client -o yaml > env-ultima.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ultima-dep
spec:
  selector:
        matchLabels:
          app: ultima-app
  template:
        metadata:
          labels:
                app: ultima-app
        spec:
          containers:
          - name: ultima-app
                image: nginx
                env:
                - name: ULTIMA
                  value: 55.55.58.23  ## value from the clusterIP
```

```
kubectl create -f env-ultima.yaml
kubectl get deployment
kubectl get pods | grep ultima-dep
kubetl exec ultima-dep-x247y -- env   ## validate
```

165. Figure out a way to create a deployment with 3 replicas using the the nginx container that can have pods deployed on the master node.

Solution:

```
sudo kubectl get nodes
sudo kubectl describe node MASTERNODE

### notice the taint on the master node:

Taints:          node-role.kubernetes.io/master:NoSchedule

### add the toleration to the yaml file

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
        matchLabels:
          app: nginx
  template:
        metadata:
          labels:
                app: nginx
        spec:
          containers:
          - name: nginx
                image: nginx
          tolerations:
          - key: "node-role.kubernetes.io/master"
                operator: "Equal"
                effect: "NoSchedule"
```

166. Copy all Kubernetes scheduler logs into a /opt/schedular-log.txt

Solution:

```
kubectl get pods --namespace=kube-system
kubectl --namespace=kube-system logs kube-scheduler-master > schedular-log.txt
```

165. Run the pod below until the counter in exceeds 30, export the log file into a file called counter-log.txt.

```
apiVersion: v1
kind: Pod
metadata:
  name: counter1
spec:
  containers:
  - name: count
        image: busybox
        args: [/bin/sh, -c, 'i=0; while true; do echo "$i: $(date)"; echo "$(date) - File - $i" >> /var/www/countlog; i=$((i+1)); sleep 3; done']
```

Solution

```
vi counter1.yml
        apiVersion: v1
        kind: Pod
        metadata:
          name: counter1
        spec:
          containers:
          - name: count
                image: busybox
                args: [/bin/sh, -c, 'i=0; while true; do echo "$i: $(date)"; echo "$(date) - File - $i" >> /var/www/countlog; i=$((i+1)); sleep 3; done']

kubectl create -f counter1.yml
kubectl exec counter1 -- cat /var/www/countlog > counter-log.txt
```

166. Create a deployment running nginx, mount a volume called "hostvolume" with a container volume mount at /tmp and mounted to the host at /data. If the directory isn't there make sure it is created in the pod spec at run time. Go into the container and create an empty file called "my-doc.txt" inside the /tmp directory. On the worker node that it was scheduled to, go into the /data directory and output a list of the contents to list-output.txt showing the file exists.

Solution:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
```

```
            run: haz-docs2
  name: haz-docs2
spec:
  replicas: 1
  selector:
            matchLabels:
              run: haz-docs2
  template:
            metadata:
              labels:
                        run: haz-docs2
            spec:
              containers:
              - image: nginx
                        name: haz-docs2
                        volumeMounts:
                        - mountPath: /tmp
                          name: hostvolume
                        ports:
                        - containerPort: 80
            volumes:
            - name: hostvolume
                        hostPath:
                          path: /data
                          type: DirectoryOrCreate
```

167. Create a namespace called "myns" in your cluster. Create a deployment called "db-deploy" that has one container running mysql image, and one container running nginx:1.7.9 in namespace "myns".
create another deployment called nginx-deploy with a single container running the image nginx:1.9.1 in namespace myns. Export the output of kubectl get pods for the myns namespace into a file called "pod-list.txt"

solution:

```
kubectl create namespace myns

kubectl -n myns create deployment db-deploy --image=mysql --dry-run=client -o yaml > db-deploy.yaml
  vi db-deploy   ## add the second container definition inside pod spec.

            apiVersion: apps/v1
            kind: Deployment
            metadata:
```

```
    creationTimestamp: null
    labels:
        app: db-deploy
    name: db-deploy
    namespace: myns
  spec:
    replicas: 1
    selector:
        matchLabels:
          app: db-deploy
    strategy: {}
    template:
        metadata:
          creationTimestamp: null
          labels:
              app: db-deploy
        spec:
          containers:
          - image: mysql
              name: mysql
          - image: nginx:1.7.9  ## add
              name: nginx       ## add
              resources: {}
  status: {}
```

kubectl create -f de-deploy.yml

kubectl -n myns create deployment nginx-deploy --image=nginx:1.9.1

kubectl -n myns get pods > pod-list.txt

168. disable the scheduling to node1 in your cluster & write the output to /opt/node-SchedulingDisabled.

Solution:

kubectl cordon node1

kubectl get nodes > /opt/node-SchedulingDisabled.txt

169. now enable the scheduling again to node1 in your cluster & write the output to /opt/node-SchedulingEnabled.txt

Solution:

```
kubectl uncordon node1
kubectl get nodes > /opt/node-SchedulingEnabled.txt
```

170. create a pod named redis using image radis:alpha

171. create a pod using image tomcat & assign label app with value web

172. create a pod named kual203 using image lerndevops/alpine:beta, find the errors in the logs, write the error messages to a file /opt/kual203.txt

173. Deploy a messaging pod using the redis:alpine image with the labels set to tier=msg & app=web

174. list all the pods with label app=web & write the output to a file /opt/webpods.txt

175. deploy pod using https://raw.githubusercontent.com/lerndevops/educka/master/exam-prep/app-redis.yml
if there are any errors try to fix them and ensure the pod status is running. DO NOT delete the pod.

176. Create a POD in the finance namespace named red-bus with the image redis:alpine

177. deploy a pod with forex name using lerndevops/tomcat:8.5.3 in finance namespace with label env=qa

178. list all pods with running status in finance namespace and write the output to /opt/kufinpods.txt

179. deploy a pod named envpod using image nginx & apply env variables username=dbuser password=dbpass

180. deploy a pod named dbpod using image lerndevops/alpine:sleep with label app=testapp, env variable uname=appuser & set the restart policy to never

181. genereate pod definition file with pod named apppod with application contianers using nginx,tomcat,redis then save the definition file to /opt/apppod.yml. then deploy the pod & ensure it is running state.

Learn & Lead IT services